



History of PS
Getting Started
Finding Help

Why use Powershell?

- Windows Administration is becoming more complicated.
 - Being able to 'get by' using the GUI is nearly impossible at an enterprise level
 - Some mainstream Windows Applications are Powershell-exclusive (Exchange, SCCM, SharePoint).
- Efficiency is limited at the GUI level.
 - You may be a 'fast typist' but you're not "automate this 10000 times a second" fast.
- The predecessors to Powershell are clunky and incomplete.
 - Batch, VBScript, etc.
- DevOps, Cross-platform, *CLI-extensibility*, *GUI-less* servers, and many more reasons.

What *is* Powershell?

- Powershell is a Command-line Interface (CLI).
- Powershell is 18 years old (November 14, 2006) as of Jan 2024
- Powershell can run command-line utilities.
- Powershell can be scripted.
- Powershell is object-oriented
- Powershell is modular.
- Powershell is *extensible*.
- Powershell is intuitive.
- Powershell is complete.

The (short) History of PS

(brief) History of PowerShell

- Originally called **Monad** (2003)
- Renamed PowerShell (1.0) and released in 2006 for:
 - Windows XP SP2
 - Windows Vista
 - Windows Server 2003
- PS 2.0 Release in 2009
 - Release for Windows 7
 - Windows Server 2008
 - Added **Remote PS** execution and 100's of commands

(brief) History of PowerShell

- Powershell 3.0 Released in 2012
 - More new features
 - Improved Integrated Scripting Environment (ISE)
 - Tab completion improved
- PowerShell 4.0 Release 2013
 - Desired State Configuration (DSC) to manage and deploy Windows services
 - Commonality of configuration is an important security principle
 - Inconsistent configuration leads to exploitation.
 - Cmdlets for user/group management
 - Deploy new Software

(brief) History of PowerShell

- Powershell 5.0 Release in (2014)
 - Available in [Windows Management Framework \(WMF\) 5.0](#)
 - Available for all Server 2016 and Windows 10 machines as of build 1809

Run down on Windows PS versions

Backwards compatibility

- Some older versions of PS do not have the commands of the newer versions
- Some scripts written in older versions will not work for the same reason.
- 2.0 is backwards compatible with 1.0
- 3.0/4.0 are backwards compatible with 2.0
- Find and set your Powershell versions
 - PS C:\> \$PSVersionTable
 - PS C:\> powershell -version #

A fork in the road...

- In 2006 Microsoft did something *crazy* (well, for Microsoft...).
- Powershell v6 was made open-source and renamed to *Powershell Core* (PS Core).
- Not only is PS Core open-source, it's also cross OS compatible with Mac OS and Linux.
- Important: we operate in an environment with TWO different versions of Powershell
 - Windows Powershell (versions 1-5)
 - Powershell Core or Powershell (version 6-7)

Notes about PS Core

- New features:
 - Universal installer packages for Linux
 - Support for Windows Microsoft Update
 - Improved tab completions
 - PSReadLine 2.1 with predictive IntelliSense
- Some commands from previous versions of PS are different or do not exist.
- Make sure you are running **PS core** when you launch PS.
- PS core does not have an ISE. We will use **VS Code**, instead.
- The book refers to PS Core as simply 'Powershell'. The old version (5.1 and older) are referred to as 'Windows Powershell'.

Getting Started

Obtaining and Installing Powershell v5.1

- Windows 10/11/Server2016 (and up) uses version v5.1 and it is installed by default.
- You can download Powershell v5.1 from <https://docs.microsoft.com/en-us/powershell>
- There are two applications included with Powershell
 - PowerShell.exe (the console)
 - PowerShell_ISE.exe (Integrated Scripting Environment)
(note: ISE is not available on non-GUI versions of Windows)

Installing PS Core

- PS Core runs *alongside* PS Version 5.1 (not in place of)
- There are differences between the two environments
- This course will use PS Core v7.4 (the current build) exclusively
- You can download the MSI package here:
<https://learn.microsoft.com/en-us/powershell/scripting/install/installing-powershell-on-windows>
- Check your installed version number with `$PSVersionTable`

Opening PS and PS_ISE v5.1

- The paths for Powershell are:
 - [C:\Windows\System32\WindowsPowerShell\v1.0Powershell.exe](#)
 - [C:\Windows\System32\WindowsPowerShell\v1.0Powershell_ISE.exe](#)
- Start Menu → Windows Powershell
- Run + Powershell (its in the run path).
- Right-Click Start (or Win + X) → Powershell

Opening PS Core

- The paths for Powershell are:
 - [C:\Program Files\PowerShell\7\pwsh.exe](#)
- Start Menu → Powershell
- Run + pwsh (its in the run path).
- ~~Right-Click Start (or Win + X) → Powershell~~

PS Integrated Scripting Environment (ISE)

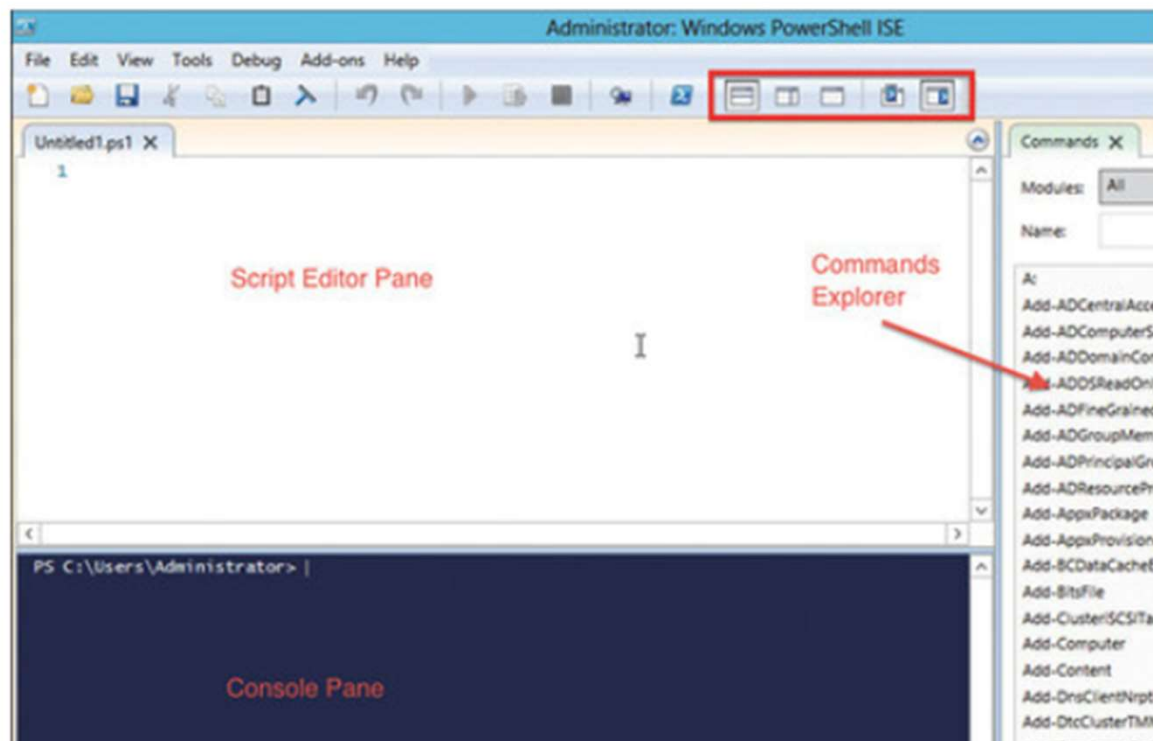
- PS ISE has many features to assist with script creation. A few things to note.

Table 2.1. ISE pros and cons

Pros	Cons
ISE is nicer looking and supports double-byte character sets.	It requires Windows Presentation Foundation (WPF), which means it can't run on a server that's had the GUI uninstalled (although it can run in Minimal Server GUI mode, which supports WPF applications).
It does more to help you create PowerShell commands and scripts, as you'll see later in this chapter.	It takes longer to get up and running, but usually only a couple of seconds longer.
It uses standard copy-and-paste keystrokes.	It doesn't support transcription in versions prior to 5.0.

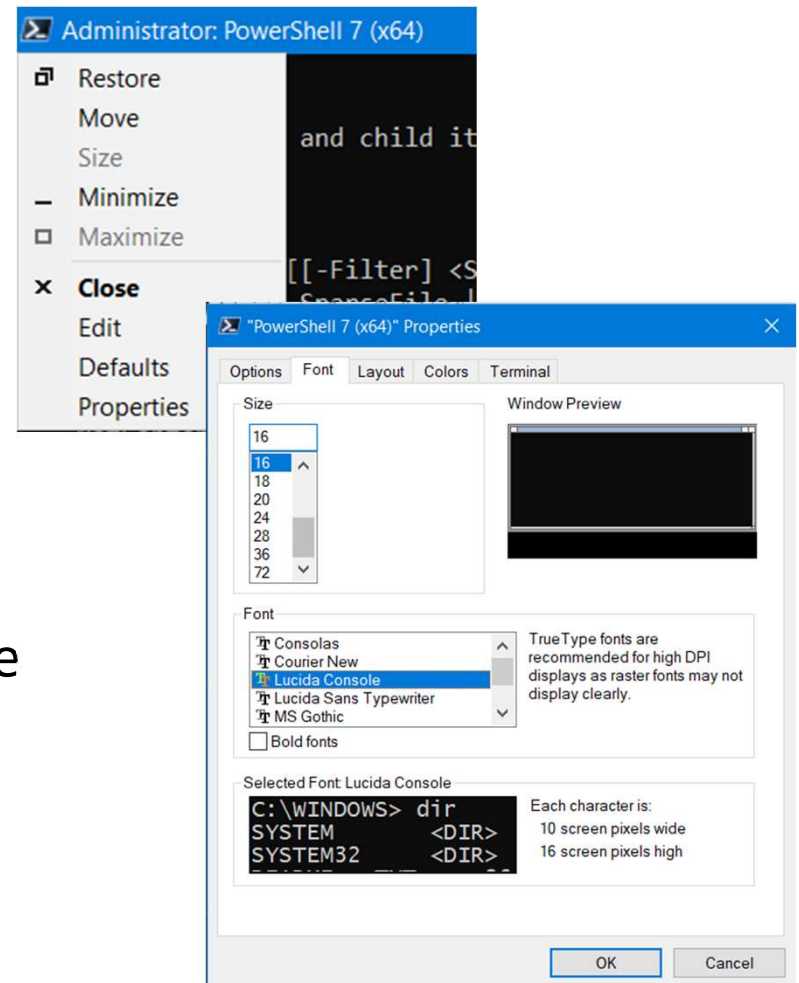
PS Integrated Scripting Environment (ISE)

Figure 2.5. The three main areas of the ISE, and the toolbar that c



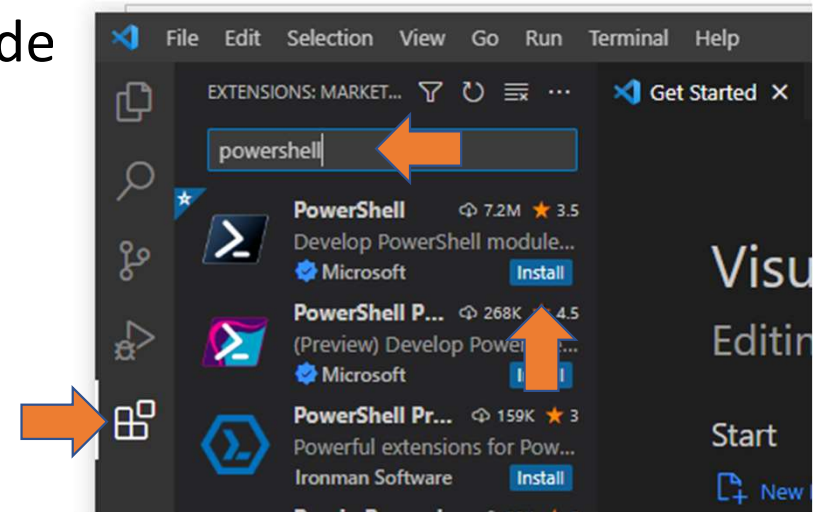
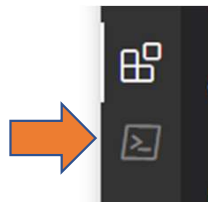
Configuring the Shell

- PS can be configured from the properties dialog
- Change the font size and type
- Change colors of certain commands or parameters (don't do this for now).
- If you bork your interface, you can use the "Defaults" option.

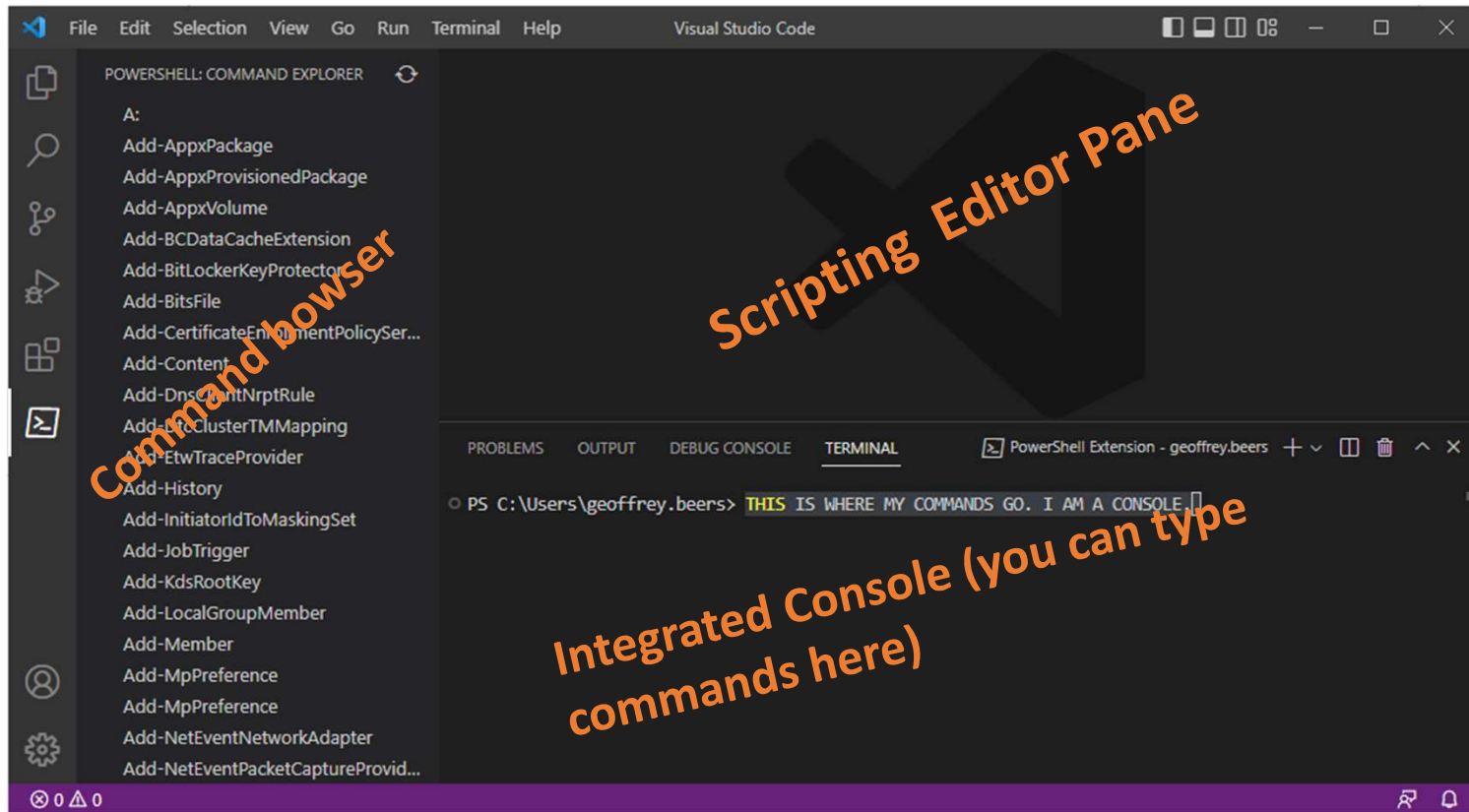


VS Code (the new ISE)

- A super-powered text editor with **Intellisense**!
- You can download the installer for VS Code here:
<https://code.visualstudio.com/Download>
- To install Powershell modules for VS Code click on the extensions link of the left, search for Powershell, and click install
- A new icon will appear on the left

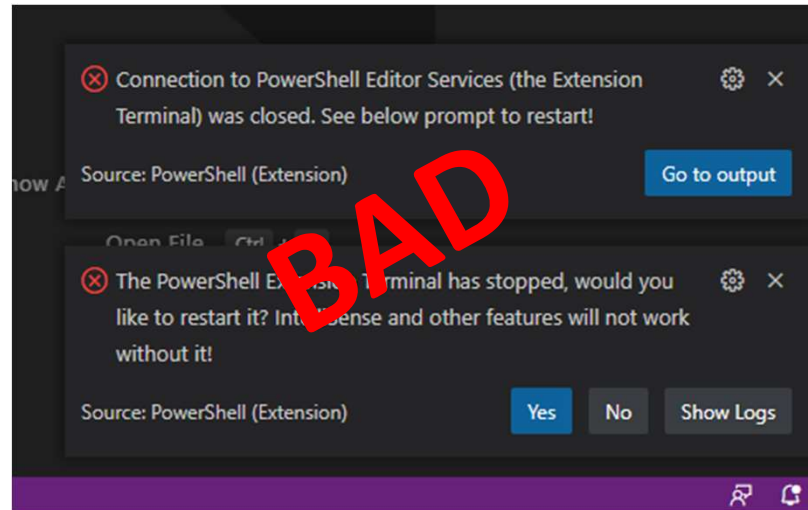
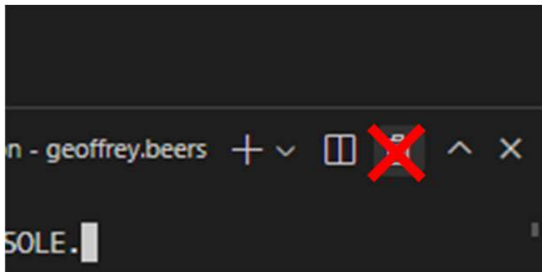


Anatomy of VS Code



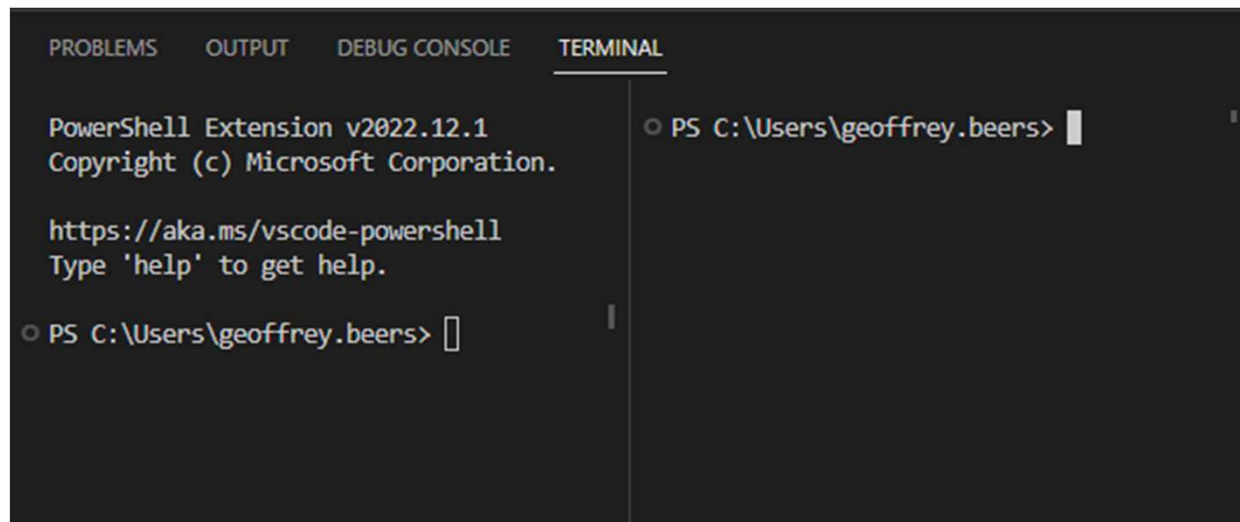
Anatomy of VS Code

- Do not click the Trash! This will kill the terminal and the extension.

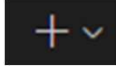


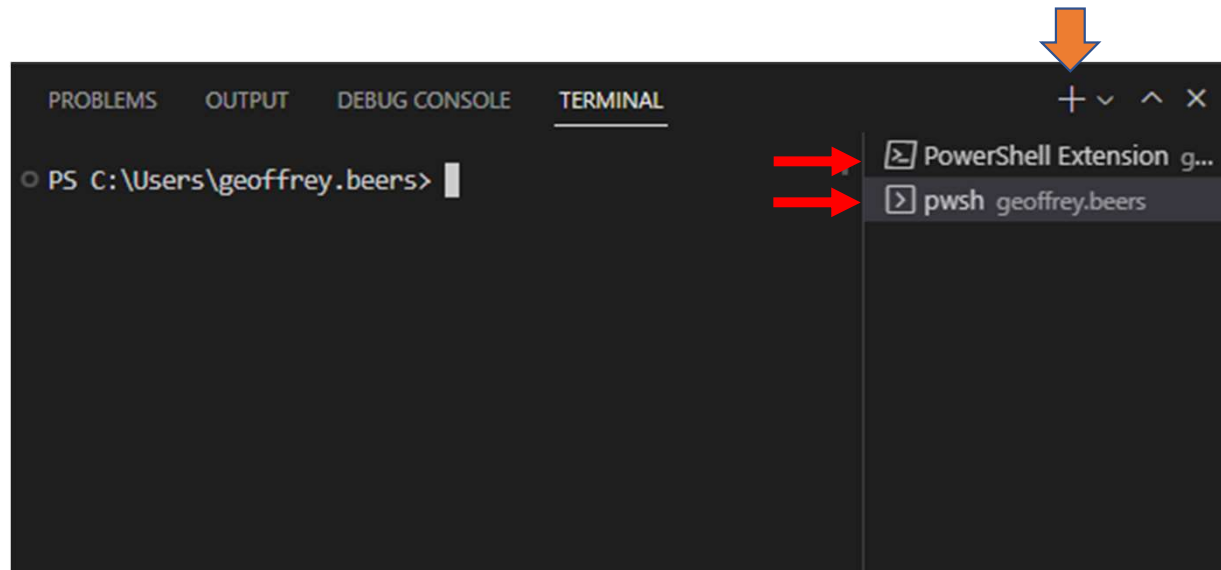
Anatomy of VS Code

- To split the terminal use the  button.
- This allows us to have two or more terminals side by side.



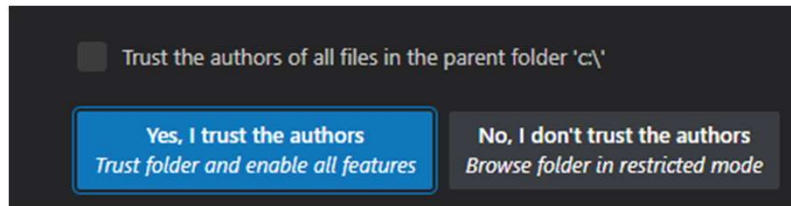
Anatomy of VS Code

- To open a new terminal (not split) use the  button.
- You can then use the browser on the right to switch back and forth between them.

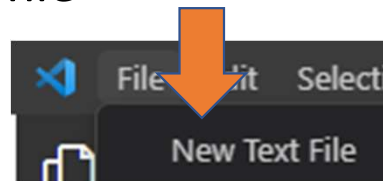


Anatomy of VS Code

- The scripting window allows you to write blocks of commands and run them in the console below.
- Step 1: Make a folder in Windows to keep your things.
- Step 2: Click  icon to open that folder.
- Step 3: Trust the folder and authors in that folder

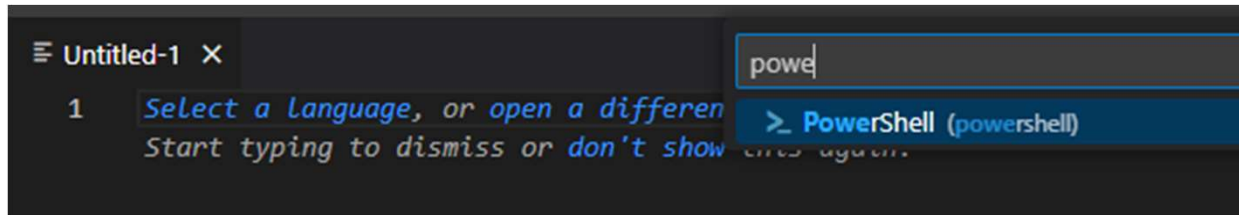


- Step 4: Make a new file



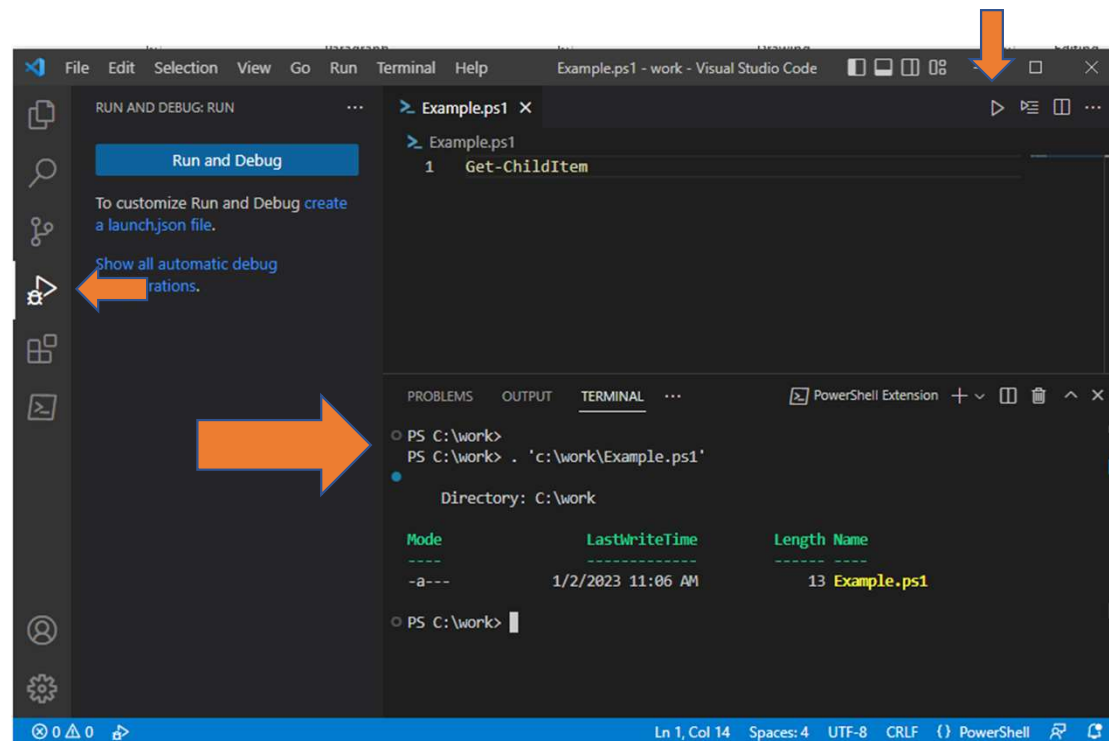
Anatomy of VS Code

- Step 5: Select the language of the file. Search for 'Powershell'.



Anatomy of VS Code

- Step 6: Run the command. The code runs in the console at the bottom.



Entering Commands (cmdlets)

Entering commands (the good news)

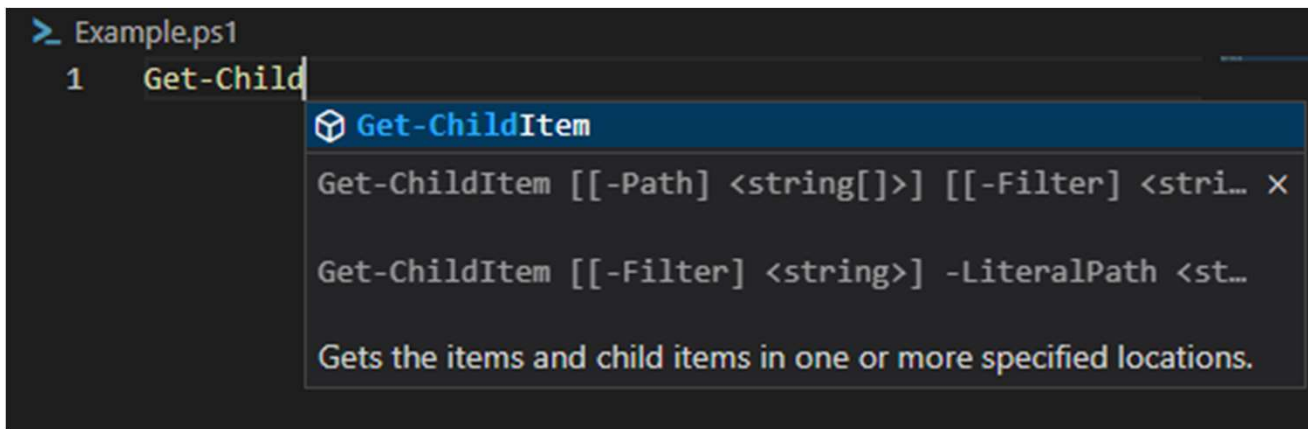
- Tab completion! You can tab complete *everything* (if it is syntactically correct, or close enough).
- You can use TAB to move through items and Shift-Tab to move backwards through items.
- The bad news: PS is still command line. PS is just as *unforgiving* of your mistakes as any other CLI. If you type a command wrong, the best-case scenario is that it simply fails.

Tab Completion

- Start typing a command and use tab to complete it or cycle through the commands that begin with those letters.
- Type a *—(dash), followed by a cmdlet name, and you can tab cycle through the verbs of that cmdlet (*-help for example).
- You can tab complete parameter names.
- You can tab complete *values* of parameters (where available).

Intellisense

- Intellisense (in VS Code) allows you to see a filtered list of commands that match the text you've entered.

A screenshot of the Visual Studio Code editor showing a PowerShell script file named 'Example.ps1'. The first line of the script is '1 Get-Child'. An Intellisense dropdown menu is open, displaying a list of PowerShell commands that match the text entered. The first item, 'Get-ChildItem', is highlighted with a blue background. Below it, two other commands are listed: 'Get-ChildItem [[-Path] <string[]>] [[-Filter] <stri... x' and 'Get-ChildItem [[-Filter] <string>] -LiteralPath <st...'. At the bottom of the dropdown, a description reads: 'Gets the items and child items in one or more specified locations.'

```
> Example.ps1
1 Get-Child
  Get-ChildItem
  Get-ChildItem [[-Path] <string[]>] [[-Filter] <stri... x
  Get-ChildItem [[-Filter] <string>] -LiteralPath <st...
  Gets the items and child items in one or more specified locations.
```

The good news: Help!

Why use help?

Two main uses of Help in PS

1. To find commands to use
2. To learn how those commands work.

PS Help System Explained

- First off: (quoting the book) RTFM!
 - Don't "dive right in". **Take some time to read**
 - Don't' like reading? Tough! **You're going to have to.**
 - Resist the urge to Google the answer, *instead*, learn **HOW** the commands work.
- The help system in PS:
 - Helps you find the commands you're looking for
 - Displays contextualized help in error messages
 - Teaches you the nuances of each command and how they can be used together.
 - Is **VERY** robust

PS Help System Explained

- If you have not opened help for a command you are using, you should, *at least once*. For EVERY command. Just to look around and *learn*.
- *Syntax* is STILL a thing in PS. You will learn syntax from the help system.
- Remember:
 - You can still google!
 - SS64.com
 - Docs.Microsoft.com
(<https://learn.microsoft.com/en-us/powershell/>)

Commands, Cmdlets, and more

- Everything that does anything in PS is a “command”.
- Powershell distinguishes between a CMDLET, a function, a method...there's more.
- You can use the term **CMDLET** and **Command** interchangeably.

Updating Help

- Help files may be, initially, incomplete. You need to update the help files.
- Help files in PS can be updated (they change over time just like any living document).
- The first time you look for help, it will take a bit to finish, as help will be updated for all installed Powershell Modules.

Updating Help

- On first install of PS, you should *always* update your help files.
- Manually update help files using the `update-help` cmdlet
 - PS must be opened Administratively.
 - If you want to update more than once per day, include the `-force` parameter.
- Help files should be updated periodically to make sure they stay up-to-date.
- We can import/export help as well (useful if you don't have an internet connection).
 - `Save-help <path>`
 - `Get-help -sourcePath <Path>`
- All help files are opensource, and can be downloaded from <https://github.com/MicrosoftDocs/PowerShell-Docs>

Getting Help

Get-help VS help VS man

- `Get-help` is a *command*
- `Help` is a *function*
 - This is actually `Get-Help <cmdlet> -full | more`
- `MAN` is an *alias* for `Get-Help`
- Typing `q` at any time will quit the `more` output.

Finding Commands with help

- **Get-help** can find **command help files** (**not** commands). If a command doesn't have a help file, **Get-help** is useless.
 - **Get-help -name <cmdlet>**
 - All commands are in a *verb-noun format*.
 - Wildcards can be used in the **verb** or the **noun**
- Example:
 - **Help *service***
 - **Help Get-Service**
- **Note:** some of the XMLs for PS help files are incomplete. If this is the case, use **Get-Help <cmdlet> -online** to view the full online help.

Get-Help Parameters

- Syntax: `Get-Help <parameter> <Value>`
- `-name` (positional parameter)
 - example: `get-help -name Get-Service`
 - example: `get-help Get-Service`
- With no other parameters, `Get-Help` will return a *summary help* files by default.

Side note: **Get-Command**

- **Get-Command** is the cmdlet used to find *actual* commands.
- **-noun** or **-verb** parameters will allow you to filter by verb/noun respectively.
- **-commandType** can be used to filter by command type (cmdlet, filter, function, etc).

Note: **-verb/-noun** and **-commandType** cannot be used in the same command.

Interpreting **Get-Help** (Parameter Sets)

- **Get-Help** will display one or more Parameter Sets
- Some commands have more than one acceptable syntax (**Parameter Set**).
- The differing Parameter Sets are *mutually exclusive* to each other.
- The Parameter Set will control the behaviour of the command.

```
NAME
    Get-Command

SYNTAX
    Get-Command [[-ArgumentList] <Object[]>] [-Verb <string[]>] [-Noun <string[]>] [-Module <string[]>]
    [-FullyQualifiedModule <ModuleSpecification[]>] [-TotalCount <int>] [-Syntax] [-ShowCommandInfo] [-All]
    [-ListImported] [-ParameterName <string[]>] [-ParameterType <PTypeName[]>] [<CommonParameters>]
    Get-Command [[-Name] <string[]>] [[-ArgumentList] <Object[]>] [-Module <string[]>] [-FullyQualifiedModule
    <ModuleSpecification[]>] [-CommandType {Alias | Function | Filter | Cmdlet | ExternalScript | Application | Script
    | Workflow | Configuration | All}] [-TotalCount <int>] [-Syntax] [-ShowCommandInfo] [-All] [-ListImported]
    [-ParameterName <string[]>] [-ParameterType <PTypeName[]>] [<CommonParameters>]
```

Parameter Set one

Parameter Set two

Optional/Mandatory Parameters.

- Some parameters are required, others are optional.
- Required parameters are indicated without any brackets.
- Optional parameters are contained within []
- Examples:
 - `CMDLET -RequiredParameter <requiredValue>` ← Named, Mandatory
 - `CMDLET [-OptionalParameter]` ← Named, Optional
 - `CMDLET [-OptionalParameter <requiredValue>]` ← Named, Optional
 - `CMDLET [-PositionalParameter] <requiredValue>` ← Positional, Mandatory
 - `CMDLET [[-PositionalParameter] <requiredValue>]` ← Positional, Optional

Abbreviating Parameters

- You only need to type enough of the parameter so that it can be interpreted by PS (not ambiguous).

- Example:

```
New-Item -type Directory "C:\Scripts"
```

VS

```
New-Item -ty D "C:\Scripts"
```

Some commands can be run with no parameters at all.

```
Get-Item ← This will prompt for input
```

```
Get-Process ← This will simply display all processes
```

Positional Parameters

- Commands can have **positional** and **named** parameters:

• `Get-ChildItem`  `[[[-Path] <string[]>] [[-Filter] <string>] [-Include <string[]>]`

Position 1 Position 2 Named

- Finding Parameter positions:

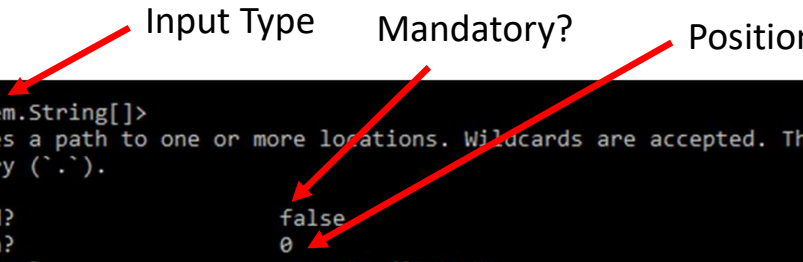
- `Get-Help <cmdlet>` (summary)
`CMDLET [-PositionalParameter] <requiredValue>`
- `Get-Help <cmdlet> -full`  This is a Switch Parameter
- `Get-Help <cmdlet> -ShowWindow`  This is a Switch Parameter

- To use positional parameters, they must be entered at the correct position
- Can be used with Named parameters providing the position is correct.
- If you specify the parameter names, they can be used out of order.
- Don't lose track of the position when writing these commands.

Positional Parameters

- **Get-Help Get-ChildItem -full**

- Will display a table that list all parameters and their position (or if they are named).



```
-Path <System.String[]>  
  Specifies a path to one or more locations. Wildcards are accepted. The default location is the current  
  directory (`.`).  
  
  Required?                false  
  Position?                0  
  Default value            Current directory  
  Accept pipeline input?   True (ByPropertyName, ByValue)  
  Accept wildcard characters? true
```

Common Parameters

- Common Parameters are parameters that exists for all commands.
- They can be included on the end of any command.
- Example: `Update-Help -Force -Verbose -ErrorAction SilentlyContinue`
- They include (aliases are in parentheses):
 - -DEBUG (db)
 - **-ERRORACTION (ea)**
 - -ERRORVARIABLE (ev)
 - -INFORMATIONACTION (infa)
 - -INFORMATIONVARIABLE (iv)
 - -OUTVARIABLE (ov)
 - -OUTBUFFER (ob)
 - -PIPELINEVARIABLE (pv)
 - **-VERBOSE (vb)**
 - -WARNINGACTION (wa)
 - -WARNINGVARIABLE (wv)

Parameter Values (<string[]>)

- Parameters without values are called *switch parameters*.
-verbose -force
- Common types of input :
(there are more than this)
 - String** – Numbers and Letters. If spaces are used, you must **single** or **double quote**.
(for now, ALWAYS use single quotes)
 - Int, Int32, Int64** – and integer (number with no decimal)
 - DateTime** – input format is based on the date/time format of your computer.

Regional format data

Select Change data formats to switch among calen formats supported by the region.

Calendar:	Gregorian Calendar
First day of week:	Sunday
Short date:	1/6/2022 
Long date:	Thursday, January 6, 2022
Short time:	11:02 AM
Long time:	11:02:58 AM

Arrays

- For `<String[]>` the `[ ]` indicates an *array*, or a list of items.
- The array can consist of 1 item (always legal), but it can be multiple items (usually separated by commas).
- Example:
 - `Get-Service -name VDS`
 - `Get-Service -name VDS,bthserv,netlogon`
- We can also reference an external list of items from a file (a text file for example) using the `Get-Content` command.
- To indicate that the 'list' is a file that should be referenced, we make another statement (inside brackets) to represent that array.
- `Get-Service -name (Get-Content input.txt)`

Finding Examples and About pages.

- Use the `-example` parameter to with help to see a list of examples.
 - `Get-help Get-Service -example`
- The `-full` parameter will (obviously) also show examples.
- The `-online` parameter can find the same help pages but in your web browser.
- “`about pages`” are supplementary for specific cmdlets.
 - Use the about pages to learn *even more* about specific commands.
 - They can be quite large but are also very detailed.
- `Help about_*` to find which about pages are available.